Practical Exam: SWD392 –SU26

Duration: 85 Minutes

Instructions

- Internet access is **NOT permitted during the exam**.
- You are allowed to use **UML modeling tools in offline mode** (as Visual Paradigm, Visio, Draw.io).

Submission Requirements

- Submit your answer as a **single Word file or a .rar/.zip archive**. Name your file exactly: **SWD392_PE_SU26_[StudentID].docx** or **SWD392_PE_SU26_[StudentID].rar**

Submissions with incorrect file names may not be graded.

Business Overview

SmartPark is an intelligent parking management system that supports customers in parking cars and motorbikes at shopping malls and office buildings.

When a vehicle enters the parking area, the system uses cameras and sensors to recognize the vehicle license plate and check the availability of parking slots. If a suitable parking slot is available, the system generates a parking ticket and automatically opens the entrance gate.

During the parking period, the system continuously updates parking slot statuses and parking durations. When customers leave the parking area, the system calculates parking fees based on parking duration and processes payment before opening the exit gate.

Parking staff members are responsible for monitoring system operations, handling incidents, and supporting customers when necessary.

The purpose of this exam is to assess your ability to design software architecture and UML models rather than implement program logic.

This examination evaluates students' understanding of software architecture and UML-based system design. Students are not required to write any source code. All answers must be presented using UML diagrams.

Students may choose either **Layered Architecture** or **Microservices Architecture**, but the chosen architecture must be applied consistently throughout all diagrams. The **Repository Pattern** is mandatory.

Students may make reasonable assumptions where necessary, as long as those assumptions do not contradict the given business context.

Question 1 (3.0 points) – Class Diagram

You are required to design a **design-level Class Diagram** for the Smart Parking system. The system is built around classes representing users, vehicles, parking tickets, parking slots, and payments. Your task is to enrich these classes with appropriate attributes, methods, and relationships.

In this system, different types of users exist with different responsibilities. Customers can own multiple vehicles, and each vehicle may generate multiple parking tickets for different parking sessions. Each parking ticket is associated with exactly one parking slot and may generate a payment transaction when the vehicle exits the parking area.

The system follows the Repository Pattern to separate business logic from data access logic.

Your Class Diagram must satisfy the following requirements:

- The classes `Customer` and `ParkingStaff` are specialized types of `User` and must inherit from it.
- A customer can own multiple vehicles.
- Each vehicle can have multiple `ParkingTicket` records.
- Each `ParkingTicket` is assigned to exactly one `ParkingSlot`.
- A parking ticket may have one payment transaction.
- Business logic related to parking ticket management and payment processing is handled by a service class, which depends on repository classes.
- Repository classes must implement a generic repository interface.

Your diagram must correctly apply UML relationships, including:

- Inheritance and interface implementation.
- Association, aggregation, and composition where appropriate.
- Dependency relationships between service and repository layers.

Multiplicity, attributes, and methods should be defined clearly and reasonably.

Question 2 (4.0 points) – Sequence Diagram

Use Case: Vehicle Entry and Ticket Generation

This question focuses on the interaction flow that occurs when a vehicle enters the parking area.

The process starts when the camera recognizes the vehicle license plate at the entrance gate. The system checks whether a suitable parking slot is available before generating a parking ticket.

If a parking slot is available, the system creates a parking ticket, updates the parking slot status, and opens the gate for vehicle entry. If the parking area is full, the system rejects access and displays an appropriate notification.

Your Sequence Diagram must meet the following requirements:

- The diagram must be designed using either Layered Architecture or Microservices Architecture.
- The interaction flow must clearly show:
  - A user-facing or input device component (Camera/Gate UI).
  - A service or controller component handling business logic.
  - Repository components responsible for data access.
  - Domain entities such as `Vehicle`, `ParkingTicket`, and `ParkingSlot`.
- Repository Pattern usage must be explicit through repository method calls.
- Conditional behavior must be modeled using `alt` fragments for:
  - Parking slot available.
  - Parking area full.

Question 3 (3.0 points) – Activity Diagram

Use Case: Vehicle Exit and Payment

This question describes the workflow that occurs when a customer exits the parking area with a vehicle.

When the vehicle arrives at the exit gate, the system identifies the parking ticket and calculates the parking fee based on the parking duration. The customer makes a payment before the system updates the parking ticket status and parking slot status.

If the payment fails, the system requires the customer to retry the transaction.

You are required to model this process using an Activity Diagram with the following characteristics:

- The diagram must include at least two swimlanes representing `Customer` and `System` responsibilities.
- Decision nodes must be used to determine whether the payment is successful or not.
- The workflow must clearly show the update of parking slot status after the vehicle exits the parking area.
- Start and end points of the activity must be clearly indicated

END OF EXAMINATION